Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность АС-68

Выполнил:
Студент 3-го курса
Группы АС-68
М.Р. Мартысюк
Проверил:
А. А. Крощенко
Доцент кафедры ИИТ,
«___»_____2026 г.

Брест 2026

Цель: построить, обучить и оценить **многослойный перцептрон (MLP)** для решения задачи классификации

Вариант 6 Диагностика диабета

- Pima Indians Diabetes
- **Задача:** предсказать наличие диабета (бинарная классификация).
- **Архитектура:**
  - входной слой;
  - два скрытых слоя: первый с 12 нейронами, второй с 8 (ReLU);
  - выходной слой с 1 нейроном (Sigmoid).
- **Эксперимент:** удалите второй скрытый слой (оставив один с 12 нейронами). Сравните recall для класса "диабет" в обеих моделях.

Код программы:

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score)

seed = 52
torch.manual_seed(seed)
np.random.seed(seed)

# 1. Загрузка и подготовка данных
columns = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
           'Insulin', 'BMI', 'DiabetesPedigree', 'Age', 'Outcome']

df = pd.read_csv("pima-indians-diabetes.csv", header=None, names=columns, comment='#')
df = df.dropna()

y = df['Outcome'].values.astype(np.int64)
X = df.drop(columns=['Outcome']).values.astype(np.float32)

print(f"Всего записей: {X.shape[0]}, из них признаков: {X.shape[1]}")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=seed, stratify=y)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

X_train = torch.tensor(X_train, dtype=torch.float32)
X_test  = torch.tensor(X_test,  dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)
y_test  = torch.tensor(y_test,  dtype=torch.float32).view(-1, 1)

N_FEATURES = X_train.shape[1]

# 2. Архитектуры моделей
class MLP_TwoHidden(nn.Module):
    def __init__(self, n_features):
        super().__init__()
        self.fc1 = nn.Linear(n_features, 12)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(12, 8)
        self.relu2 = nn.ReLU()
```

```
        self.fc3 = nn.Linear(8, 1)

    def forward(self, x):
        x = self.relu1(self.fc1(x))
        x = self.relu2(self.fc2(x))
        x = self.fc3(x)
        return x

class MLP_OneHidden(nn.Module):
    def __init__(self, n_features):
        super().__init__()
        self.fc1 = nn.Linear(n_features, 12)
        self.relu1 = nn.ReLU()
        self.fc3 = nn.Linear(12, 1)

    def forward(self, x):
        x = self.relu1(self.fc1(x))
        x = self.fc3(x)
        return x

# 3. Функции обучения и оценки
def train_model(model, epochs=200, lr=0.001):
    criterion = nn.BCEWithLogitsLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    for _ in range(epochs):
        model.train()
        y_pred = model(X_train)
        loss = criterion(y_pred, y_train)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    return model, loss.item()

def evaluate_model(model, loss_val):
    model.eval()
    with torch.no_grad():
        logits = model(X_test)
        probs = torch.sigmoid(logits).numpy().ravel()

    y_pred = (probs >= 0.5).astype(int)
    y_true = y_test.numpy().ravel().astype(int)

    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, pos_label=1, zero_division=0)
    rec = recall_score(y_true, y_pred, pos_label=1, zero_division=0)
    f1  = f1_score(y_true, y_pred, pos_label=1, zero_division=0)

    return {"accuracy": acc, "precision": prec, "recall": rec, "f1": f1, "loss":
loss_val}

# 4. Запуск эксперимента
print("\nОбучение моделей (200 эпох):")
print("Модель A: Вход -> 12 (ReLU) -> 8 (ReLU) -> 1 (Logits)")
print("Модель B: Вход -> 12 (ReLU) -> 1 (Logits)")

model_A = MLP_TwoHidden(N_FEATURES)
model_A, loss_A = train_model(model_A)
res_A = evaluate_model(model_A, loss_A)

model_B = MLP_OneHidden(N_FEATURES)
model_B, loss_B = train_model(model_B)
res_B = evaluate_model(model_B, loss_B)

# 5. Сравнение результатов
print("\n--- Результаты на тестовой выборке ---")
print("Модель           | Accuracy | Precision | Recall | F1-score")
print("-" * 65)
print(f"2 слоя (12-8)    | {res_A['accuracy']:.4f}    | {res_A['precision']:.4f}     |
{res_A['recall']:.4f} | {res_A['f1']:.4f}")
```

```
print(f"1 слой (12)      | {res_B['accuracy']:.4f}    | {res_B['precision']:.4f}     |
{res_B['recall']:.4f} | {res_B['f1']:.4f}")

print("\n--- Изменение метрик (Модель B минус Модель A) ---")
for k, title in [("accuracy", "Accuracy"), ("precision", "Precision"), ("recall",
"Recall"), ("f1", "F1-score")]:
    diff = res_B[k] - res_A[k]
    print(f"Δ {title:<10}: {diff:+.4f}")

print("\nАнализ эксперимента:")
d_rec = res_B["recall"] - res_A["recall"]
if d_rec < 0:
    print(f"Удаление второго слоя СНИЗИЛО Recall на {abs(d_rec)*100:.2f} п.п. (модель
хуже находит больных).")
elif d_rec > 0:
    print(f"Удаление второго слоя ПОВЫСИЛО Recall на {d_rec*100:.2f} п.п. (модель
лучше находит больных).")
else:
    print("Удаление второго слоя не изменило Recall.")

print(f"Финальный loss Модели A: {loss_A:.4f}")
print(f"Финальный loss Модели B: {loss_B:.4f}")
```

Результат:

```
Всего записей: 768, из них признаков: 8

Обучение моделей (200 эпох):
Модель A: Вход -> 12 (ReLU) -> 8 (ReLU) -> 1 (Logits)
Модель B: Вход -> 12 (ReLU) -> 1 (Logits)

--- Результаты на тестовой выборке ---
Модель          | Accuracy | Precision | Recall | F1-score
-----------------------------------------------------------
2 слоя (12-8)   | 0.8052   | 0.7500    | 0.6667 | 0.7059
1 слой (12)     | 0.7532   | 0.6667    | 0.5926 | 0.6275

--- Изменение метрик (Модель B минус Модель A) ---
Δ Accuracy  : -0.0519
Δ Precision : -0.0833
Δ Recall    : -0.0741
Δ F1-score  : -0.0784

Анализ эксперимента:
Удаление второго слоя СНИЗИЛО Recall на 7.41 п.п. (модель хуже находит больных).
Финальный loss Модели A: 0.4713
Финальный loss Модели B: 0.5132
```

В базовой модели (два слоя 12-8) точность составила 0.8052, а полнота (recall) — 0.6667. При удалении второго слоя обе метрики просели: accuracy до 0.7532, recall до 0.5926 (минус 7.41 п.п.). Одновременно вырос train loss (0.5132 против 0.4713). Это означает, что второй слой был необходим для улавливания сложных связей в данных, а его удаление привело к недообучению сети и ухудшению поиска больных диабетом.

Вывод: освоил построение, обучение и оценку многослойного перцептрона (MLP) для решения задачи классификации.